

Documentation Base Wavely

Introduction

Objectif :

Organiser vos photos de voyages et de vacances pour toujours vous y retrouver !
Cette application se veut comme une cellule de mémoire visuelle, où vous pouvez stocker, organiser et partager vos souvenirs.
Exportez vos photos et albums facilement pour les transmettre à vos proches.

Contexte

À l'ère où les réseaux sociaux sont omniprésents, les plateformes comme Instagram ou Facebook mélangent publications personnelles, publicités et contenus sponsorisés. De plus en plus d'utilisateurs ressentent le besoin de partager leurs souvenirs de voyage dans un espace plus intime, sans algorithme, sans publicité ni likes.

Par ailleurs, les solutions classiques de stockage (Google Photos, iCloud, Drive...) manquent de personnalisation et d'options communautaires.

Il n'existe pas encore de solution simple, intégrée et abordable pour offrir une alternative dédiée aux souvenirs de voyage.

Ce projet souhaite répondre à ce besoin.

Solutions technique

Pour la partie back-office et front office, les mêmes technologies seront utilisées afin de garantir une cohérence globale.

Le front-end sera développé sans Framework, en **HTML5**, **CSS3** et **JavaScript** pur. Ce choix permet un contrôle total du rendu et une application légère, facile à maintenir et à personnaliser.

Le back-end s'appuiera sur **PHP**, un langage largement supporté et bien adapté aux besoins du projet. Il assurera la gestion des utilisateurs, des sessions, des droits et la communication avec la base de données.

Les données seront stockées dans une **base MySQL**, fiable et performante, qui contiendra les informations essentielles telles que les comptes utilisateurs, les albums, les photos ou encore les groupes.

Ce socle technique simple et éprouvé permet de créer une application stable, accessible et facile à déployer.

BDD

[BDD](#)

Objectifs et concept

Avec cette base de donnée il devaient être possible de permettre à l'utilisateur une certaine liberté. Il faillaient qu'il puissent appartenir à plusieurs groupe, avoir plusieurs album, ajouter plusieurs photo, faire plusieurs commentaire , etc.

C'est pour cette raison que le *many to many* à beaucoup été utilisé. Cela permet une certaine maniabilité.

Avec ce concepts il a fallu utiliser *des tables de jointures* à plusieurs reprise. Comme pour les tags et les gestion de permission sur les photo ainsi que pour les group.

Fonctionnement

Utilisateur (User)

La table utilisateur est un peu le point de départ de cette bases. Il dispose d'une identité unique avec un *email* et un *mot de passe* pour se connecter. Son *username* est là comme pseudo, ce que les autres utilisateurs pourront voir.

Album

Les album sont tout aussi important car un utilisateur sera obligé d'avoir des albums pour mettre des photos.

Un album peut être liée à un *Tag* via *album_tag* qui relie les deux.

Il peut également être liée, aussi en *many to many* à *access_group_album* pour devenir un album partagé avec un groupe. Pour plus de spécificité voir [Groupe](#).

Evidement chaque album à un propriétaire qui à tout les droits dessus (suppression, modification, gestion des photo).

Photo

La table photo est fondamentale, elle est liée à plusieurs chose importantes. Comme les *commentaire*, les *Tag* et évidemment les *album*.

Pour Le champs *image_url*, il sera rentrée le dossier *uploads* suivie de l'url de l'image qui sera rendu unique au paravent depuis le coté serveur à la création.

Le champs *location* permettra d'identifier chaque photo à une destination de voyage.

Une photo a forcément un album liée, mais elles peut en avoir plusieurs. C'est à dire quelle peut faire partis de l'album "Voyage à Mykonos" et en même temps de l'album "Eté 2025". Toujours dans cette logique de donner de la liberté à l'utilisateur. C'est pour ça que la table de jointures *photo_album* est là.

Photo rôles

Cette table sert à pouvoir attribuer à un utilisateur des droits particulier sur une photo précisément, dans la continuité de donner de la liberté à l'utilisateur.

Par exemple : Vous avez un album "Entièrement de vie de jeune fille en Thaïlande". Votre mère est sur l'application. Vous voulez lui partager une photo de de l'hôtel. Or elle n'a pas besoin de voir toutes les photo de cette soirée. Donc sans lui partagé l'album, vous pouvez lui partagé la photo, et elle pourra la commenter par exemple.

Donc un utilisateur peut être `viewer` ou `owner` sur une photo.

- `owner` : Il peut voir, commenter, modifier la description, les tags, la supprimer et la partager.
- `viewer` : Il peut la voir et la commenter. Rien de plus.

Commentaire

Chaque commentaire est liée à une photo et a un auteur. Il possède également la date de la création, ce n'est pas une heure mais bien une date.

Tag

La table Tag est faite pour être adapté autant au album qu'au photo. C'est pour ça qu'elle est reliée à des table de jointure, qui font le pongs entre elle et les ressources concerné. C'est une table simple qui ne possède qu'un `id` et un `name` .

Groupe

La logique de est découpé en 3 table distincte. Le but d'un group c'est de regrouper des gens autour d'albums.

- Chaque group à un propriétaire, donc la table `group_entity` est là. On peut dire que cette table représente en quelque sorte le group.
- Comme chaque group à des membres, la table `access_group_members` les réunis. Donc en bases, il y aura une entrée pour chaque lien group/personne. Un user pourra avoir plusieurs entrée à son id puisque qu'il peut faire partis de plusieurs groupe à la fois. Et on précise le rôle pour paramétrer les droits du user sur les photo des album du groupe. Il peut être `viewer` ou `editor` .
 - `viewer` : Voir et commenter le contenu de l'album.
 - `editor` : Il peut modifier la modifier, changer les tags, la description et son nom. Mais il ne peut pas la supprimer.
- Chaque group a un ou des album . s, donc la table `access_group_album` relie le group à un album.

Le tout tient au propriétaire, donc le propriétaire à tout les droits sur son groupe (suppression, création d'album, ajout de membre, modification des droits).

Le group n'a pas de paramètre `visibility` parce que il ne sera e toute façon visible que par les

membres du groupes. Alors ce sont les albums du groupe qui détermine si ils sont visible de tous, et c'est les propriétaire qui détermine ça.